UNITÀ 1 · CODIFICA DELLE INFORMAZIONI

Simulazione della verifica Unità 1

Stessa struttura e stessa difficoltà della verifica vera, con domande diverse. Fai la prova in 55 minuti senza appunti e senza calcolatrice, poi correggiti con le soluzioni in fondo.

Nome e cognome | Classe | Data

1. Scelta multipla

15 punti · 2,5 per domanda

Segna una sola risposta per ogni domanda.

a) La parte della CPU che esegue i calcoli aritmetici e logici è…

- ○ l'unità di controllo
- ○ il program counter
- ○ la ALU
- ○ il bus dati

b) Il programma che traduce tutto il codice sorgente prima di eseguirlo si chiama…

- ○ compilatore
- ○ interprete
- ○ driver
- ○ firmware

c) Quanti valori diversi si rappresentano con 10 bit?

- ○ 1000
- ○ 1023
- ○ 100
- ○ 1024

d) Su 8 bit in complemento a 2, il byte 1111 1111 vale…

- ○ 255
- ○ −1
- ○ −127
- ○ −128

e) In Java, `Integer.toBinaryString(10)` restituisce…

- ○ il numero 2
- ○ "10"
- ○ "1010"
- ○ "A"

f) Quanti byte occupa in UTF-8 il carattere «è»?

- ○ 2
- ○ 1
- ○ 3
- ○ 4

2. Vero o falso

8 punti · 1 per frase

V ☐ **F** ☐ Il driver permette al sistema operativo di dialogare con una periferica.
V ☐ **F** ☐ La RAM conserva i dati anche a computer spento.
V ☐ **F** ☐ In base 5 esiste la cifra 5.
V ☐ **F** ☐ Il numero esadecimale 10 vale 16 in decimale.
V ☐ **F** ☐ Su 8 bit in complemento a 2 il numero più grande è 127.
V ☐ **F** ☐ Un double memorizza 0,1 in modo esatto.
V ☐ **F** ☐ I primi 128 code point di Unicode coincidono con il codice ASCII.
V ☐ **F** ☐ In Java, `(int) 3.9` vale 4.

3. Conversioni

12 punti · 1,5 per conversione

Il pedice indica la base. Mostra i calcoli nello spazio sotto la tabella.

Numero	Converti in	Risultato	Numero	Converti in	Risultato
11001011_2	decimale		101111010_2	esadecimale	
150_{10}	binario		175_{10}	esadecimale	
$2BC_{16}$	decimale		2102_3	decimale	
644_8	binario		$7B_{16}$	ottale	

4. Interi relativi in complemento a 2 (8 bit)

15 punti

a) Scrivi in complemento a 2 su 8 bit: -37 = _____ -90 = _____ (4 punti)

b) Quanto vale in decimale il byte `1111 0001`? _____ (3 punti)

c) Calcola in binario 50 + (-37), con i riporti, e verifica il risultato in decimale. (4 punti)

d) Calcola in binario -100 + (-50). Che risultato ottieni su 8 bit? C'è overflow? Perché? (4 punti)

5. Le variabili nel debugger

12 punti · 2 per variabile

```
int a = 11;
int b = 4;
double m = a / b;
double n = a / 4.0;
int t = (int) -2.5;
char c = (char) ('A' + 3);
boolean ok = (n > m);
byte x = (byte) 129;
```

Scrivi il valore di ogni variabile alla fine dell'esecuzione e, quando serve, il perché.

Variabile	Valore	Perché
m		
n		
t		
c		
ok		
x		

6. Numeri reali: IEEE 754

10 punti

a) Codifica **5,75** come float IEEE 754 a 32 bit: numero in binario, forma normalizzata, segno, esponente (con l'eccesso 127), mantissa e risultato in esadecimale. (8 punti)

S = ___ Esponente = _____ Mantissa = _____
Esadecimale = _____

b) Perché un double è più preciso di un float? (2 punti)

7. Caratteri: ASCII, Unicode, UTF-8

10 punti

a) Decodifica questi byte ASCII scritti in esadecimale: `43 6F 64 65` → _____ (2 punti)

b) Codifica in UTF-8 la stringa **«Già €»**: per ogni carattere scrivi i byte in esadecimale, poi il numero totale di caratteri e di byte. Ricorda: à = U+00E0, € = U+20AC. (6 punti)

c) Un programma ordina queste parole confrontando i codici ASCII: **Zeta, 3D, alba, Alba**. Scrivi l'ordine che ottiene. (2 punti)

8. Completa

8 punti · 1 per spazio

1. La parte della CPU che fa i calcoli si chiama _____.
2. Il compilatore Java si avvia con il comando _____.
3. Una cifra esadecimale corrisponde a _____ bit.
4. Con `chmod 750` gli "altri" hanno i permessi _____ (scrivili con le lettere r, w, x, -).
5. In un float IEEE 754 la mantissa occupa _____ bit.
6. In UTF-8 un carattere occupa da 1 a _____ byte.
7. Nel debugger, per eseguire solo la riga corrente si usa il comando _____.
8. Per il produttore di un disco, 1 TB = 10 elevato a _____ byte.

9. Domanda aperta

10 punti

Spiega la differenza tra Unicode e UTF-8, con un esempio, e perché un'emoji occupa più byte della lettera A. Racconta poi un caso in cui un errore di codifica dei numeri ha avuto conseguenze reali (Ariane 5, Patriot, YouTube o un altro). Usa almeno quattro termini studiati.

Punti: _____ / 100

Voto: _____

Soluzioni

Correggi la tua prova, somma i punti e dividi per 10: ottieni il voto in decimi. Per ogni errore torna alla lezione su quell'argomento e rifai l'esercizio.

1. Scelta multipla · 2,5 punti ciascuna

a) **la ALU** · b) **compilatore** · c) **1024** (2^{10}; 1023 è il valore massimo) · d) **−1** · e) **"1010"** (una stringa con le cifre binarie) · f) **2** (U+00E8 sta tra U+0080 e U+07FF: byte C3 A8)

2. Vero o falso · 1 punto ciascuna

1 **V** · 2 **F** (la RAM è volatile) · 3 **F** (cifre da 0 a 4) · 4 **V** · 5 **V** · 6 **F** (0,1 è periodico in binario) · 7 **V** · 8 **F** (il cast tronca: 3)

3. Conversioni · 1,5 punti ciascuna

Esercizio	Risultato	Esercizio	Risultato
11001011_2 → decimale	**203**	101111010_2 → esadecimale	**17A**
150_{10} → binario	**10010110**	175_{10} → esadecimale	**AF**
$2BC_{16}$ → decimale	**700**	2102_3 → decimale	**65**
644_8 → binario	**110100100**	$7B_{16}$ → ottale	**173**

Passaggi chiave: 2BC = 2·256 + 11·16 + 12; 2102_3 = 2·27 + 1·9 + 0·3 + 2; 7B = 0111 1011 → 1 111 011 → 173.

4. Complemento a 2 · 15 punti

a) 37 = 0010 0101 → inverti 1101 1010 → +1 = **1101 1011**; 90 = 0101 1010 → 1010 0101 → **1010 0110**. b) −128 + 64 + 32 + 16 + 1 = **−15**. c) 0011 0010 + 1101 1011 = 1 0000 1101: si scarta il nono bit, **0000 1101 = 13**. d) 1001 1100 + 1100 1110 = 1 0110 1010 → **0110 1010 = +106**: **overflow**, perché due negativi hanno dato un risultato positivo (−150 è minore di −128).

5. Le variabili · 2 punti ciascuna

m = **2.0** (11 / 4 è divisione intera = 2) · n = **2.75** · t = **−2** (il cast tronca verso lo zero) · c = **'D'** (65 + 3 = 68) · ok = **true** · x = **−127** (129 − 256)

6. IEEE 754 · 10 punti

a) 5,75 = 101,11 = 1,0111 × 2^2. S = **0**; esponente 2 + 127 = 129 = **1000 0001**; mantissa **0111** seguita da 19 zeri; 0100 0000 1011 1000 0000… = **40B80000**. b) Ha più bit: 52 di mantissa invece di 23 (e 11 di esponente invece di 8), quindi circa 16 cifre decimali affidabili invece di 7.

7. Caratteri · 10 punti

a) **Code**. b) G = 47, i = 69, à = **C3 A0**, spazio = 20, € = **E2 82 AC**: **5 caratteri, 8 byte**. c) **3D, Alba, Zeta, alba** (cifre 48–57, maiuscole 65–90, minuscole 97–122).

8. Completa · 1 punto ciascuno

ALU · **javac** · **4** · **---** · **23** · **4** · **step over** · **12**

9. Domanda aperta · 10 punti

Risposta attesa: Unicode assegna a ogni carattere un numero, il **code point** (A = U+0041, € = U+20AC); **UTF-8** stabilisce come scrivere quel numero in **byte**, da 1 a 4. La A sta nella fascia ASCII e occupa 1 byte; un'emoji ha un code point oltre U+FFFF e occupa 4 byte, a volte di più se è una sequenza (modificatori, ZWJ). Esempio di caso reale: Ariane 5 (1996), una velocità convertita in un intero con segno a 16 bit andò in **overflow**; oppure il Patriot (1991), dove 0,1 non era rappresentabile esattamente e l'errore si accumulò. Griglia: Unicode e UTF-8 distinti con esempio 4 punti; byte dell'emoji 2 punti; caso reale corretto 2 punti; termini e chiarezza 2 punti.